THE OPEN SOURCE AUDIT

A Structured Examination of Python

Chosen Software:

Python 3.x — PSF License

Course: Open Source Software | VITyarthi

Submitted By:
Name: *Devesh Choudhari*
Registration Number: *24BEC10091*

Program: BTech

Branch: *Electronics and Communication Engineering*
Semester: *V*

Submitted To:
Faculty Name: Dr Pradeep Kumar Mishra
Course Code: *CSE0002*

Institution:
VIT Bhopal University

March 2026

License	First Released
Python Software Foundation License	1991 by Guido van Rossum

Table of Contents

1. Introduction 3
2. Part A1 — Origin Story and Problem Context 4
3. Part A2 — License Analysis 6
4. Part A3 — Ethics of Open Source 8
5. Part B — Linux Footprint 11
6. Part C — The FOSS Ecosystem 13
7. Part D — Open Source vs Proprietary 15
8. Shell Scripts 17
9. References 20

1. Introduction

Many programs that help run today's world are not made by companies and sold to make money. They are created openly by many people, who use them together, and they are developed together. This is what the open-source philosophy believes — that knowledge should not cost money to access, tools should be general-purpose, and groups of people can create things that last much longer than any one company or institution.

This report is an examination of Python — one of the most popular programming languages in the world — as it relates to the open-source philosophy. Python is more than software; it also provides a way to see and understand how people can work together to create software and technology through collaborative processes (i.e. open collaboration). Through thoughtful licensing and a strong community, Python and its tools effectively impact industries ranging from scientific research to artificial intelligence (AI) to web development and ultimately education.

The report is divided into four sections that will help explain how open-source technology relates to Python. A) Provides an overview of Python's history, its licensing, and the ethical issues raised by open-source technology; B) Shows how Python is used on a Linux operating system; C) Shows the communities and tools that support Python; D) Provides a critical analysis showing how Python compares with its leading proprietary competitor, MATLAB.

There are Five Shell scripts included with this report; each scripting example provides evidence of the Linux skills and open source toolsets covered and include documentation such as the code for each script, as well as the expected output from running each script, and a description of the shell scripting concepts utilized in each script.

This is not a review of Wikipedia; rather, it is an attempt to explore in detail what led to a single individual sitting in the Netherlands in 1989, had reached a point of frustration with existing software and tools and decided to develop a program and make it available free of charge to others. And how this decision continues to be significant today.

2. Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

To comprehend how the birth of Python came about, let’s think back to how programming was done back in the late ‘80s. At this point in time, several different languages dominated programming such as C, Fortran, and Pascal. All were powerful languages, they required skillful attention to detail. It was necessary to manage memory, declare types, compile programs, and take care of other manual tasks before a single program could run successfully. These languages rewarded experience and punished beginning programmers heavily.

In contrast, there was shell scripting and utility programs such as AWK or Perl. They had more flexibility and could produce results quickly but were not able to process complex logic. Perl, for example, was often described as difficult to read. The prevalent joke among software development communities during this period stated that Perl code resembled the jumbling of characters produced by an old modem. It worked but would take weeks to decipher after it ran.

The Frustration at the Centre

Guido van Rossum, a programming expert working at Centrum Wiskunde & Informatica in Holland, was involved with developing a programming language called ABC (a programming language designed to be easy to read/use). ABC had a few wonderful ideas that inspired Guido, such as a clean syntax, indentation used as a structural element, and no large repetitive sections of fixed code (which made ABC very clear).

The challenge he faced was ABC was not open. ABC's developers tried to control the absolute majority of the functionality in ABC, thus preventing anyone from expanding on the functionality of ABC. Because of this lack of openness, ABC never left the skin of the lab.

The other programming language/environment that Guido used extensively was Unix. Although Unix has many advantages compared to other language environments, there were some obstacles that Guido encountered regarding the difference between Perl scripts and C programs (how easy/difficult it was to produce a Perl script versus producing a C program) when creating "real" programs using Unix systems. There was no middle ground that bridged the gap between being an intermediary programmer (user of shell scripts) and a "real" programmer (C programmer).

As one programmer stated, "When you are using a shell script to perform some type of task, you can produce code quickly, and it performs well under most circumstances. However, once the task becomes complicated, performing any type of task using shell scripts becomes far more difficult. On the other hand, producing C programs takes much more time and is very difficult to debug if you are using C to develop programs that require an exploratory environment. There is literally no middle ground between the two for producing software."

The Christmas Holiday and a New Language

In December of 1989, when the CWI offices were closed for the holidays, Guido van Rossum began programming what would later on become known as Python. He selected this name from Monty Python's Flying Circus because: (1) it wasn't related to animals and (2) he wanted a short, distinctive, and somewhat irreverent name to represent his creation.

Guido had several goals he set out to achieve with Python. He wanted it to be easy to read, even for non-programmers; he wanted indentation to provide structure to code; he wanted a simple interface to the C programming language so that Python could act as a connective element between other more powerful and lower-level programming languages; last but not least, he wanted this product to be open, in that it was not only functional, but it also allowed for modifications, in addition to being available to anyone and anyone could extend.

In February of 1991, the first official version of Python (version 0.9.0) was released at the alt.sources newsgroup (this post was archived). There was an overwhelmingly positive response from developers who also felt that Guido's assessment of the existing state of software development was correct, and they found that not only was Python going to be the answer to their problems, but they also had found a language that felt familiar to them, and would be able to support their projects over an extended period of time and also had no restrictions on how they could use the product (i.e., no licensing fees).

Why Open, Why Shared

Guido's decision to openly distribute Python was not arbitrary. He had experienced how closed-source software negatively impacted ABC. He had learned through his experience working with others at CWI and the larger Unix community that open-source software is an asset in maintaining quality software. By posting Python to the newsgroup, he welcomed constructive feedback and offered opportunities for developers to contribute to the expansion of Python's capabilities. As a result, Python's first user community began immediately to enhance the system.

Python is able to address the needs of a broad range of users, and is able to do so through an open-source process with an inclusive nature. The initial problem that Python was designed to solve is very specific; however, the method in which Python was originally developed and sold is equally as significant as the language itself. Guido's decision to trust in the open-source community and allow the community to participate in Python's development was ultimately the motivation behind the creation of the programming language itself.

3. Part A2 — The License: What It Actually Says

Python is released under PSF License (The Python Software Foundation License) an open-source, permissive license type that has been approved by the Open Source Initiative. To fully appreciate the nature of this license and how it can be utilized practically, it is useful to start with Richard Stallman's GNU Project's four freedoms of free software.

The Four Freedoms of Free Software

Freedom 0	The freedom to run the program as you wish, for any purpose.
Freedom 1	The freedom to study how the program works, and change it so it does your computing as you wish. Access to the source code is a precondition for this.
Freedom 2	The freedom to redistribute copies so you can help others.
Freedom 3	The freedom to distribute copies of your modified versions to others, so the whole community can benefit from your changes.

The permissions granted by a PSF license, as per the PSF's 4 "freedoms," allow people to use, read, redistribute, and modify (with some restrictions) any python version.

The PSF license allows anyone to use python for any purpose (commercial or personal).

You can view the complete source code of CPython (the default implementation of python) on GitHub.

You can freely redistribute python.

You may modify and redistribute your modified version of python as long as you include the psf license notice in the modified version and do not use the python software foundation's name to endorse your product without the psf's permission.

Can a Company Take Python, Modify It, and Sell It?

This is arguably the most significant practical difference that exists between PSF and GPL; on the one hand, the GPL has a "copyleft" requirement with regard to distribution of modified works when you transfer them from your own copy of the software to another copy of the software; that is, if you modify and distribute GPL code (or code derived from GPL code) you must also provide your changes under the same license. The PSF has no such requirement; any entity may take the Python source code as provided by the PSF, modify it for their purposes, and create a proprietary product. The company may sell this product without providing any of the changes back to the community.

There are many examples of this in the real world. For example, Intel, Google, and many others have created modified versions of Python for internal usage. They have very little, if any, obligation to give back to the community; they just have to leave the copyright notice and PSF license text intact. There are no other obligations. This was a conscious decision by Guido and PSF; they

decided that they would prefer to have Python widely adopted than to enforce their rights. The result has been that Python is now everywhere, because no one has to worry about using it.

GPL vs MIT vs PSF: A Comparison

Feature	GPL v3	MIT	PSF (Python)
Share source of changes?	Required (copyleft)	Not required	Not required
Commercial use allowed?	Yes, but changes must stay open	Yes, freely	Yes, freely
Patent protection?	Explicit grant	Implied only	Not explicit
Complexity	High	Very low	Low

Which Would I Choose, and Why?

I would select an MIT license if I were developing software for maximum adoption (a library or framework). The MIT license is an extremely simple and permissive license, in that it doesn't place any restrictions on use or distribution of the software. Therefore, the MIT license encourages large-scale use of code and technology by developers and the general public. If I want my work to be freely available to all (as in the case of GPL), but also want to prevent someone from taking my work and closing it down, I'd select the GPL license. The PSF license for Python has also followed this trend of permissive licensing throughout the development of Python and has benefited Python tremendously.

Free as in Beer vs Free as in Freedom

The term 'Free as in beer' refers to the fact that there is no cost associated with downloading or using the software, while 'Free as in freedom' refers to the freedoms one has to use, study, change, and share the software. Python is both free from a cost perspective and free from a freedom perspective – when you download Python, you won’t have to pay anything. However, even more importantly, you can read, change, build-on and redistribute the complete source code of CPython, so you have total freedom to do whatever you want with it. When a university in Africa installs Python, they are not only getting the software for free; they are also getting a tool to modify as needed, translate, add to, and teach others how to use it, which is an even greater meaning of free.

4. Part A3 — The Ethics of Open Source

Should All Software Be Open Source?

The Case For

There are many reasons to argue that all software, especially software which controls critical infrastructure (hospitals, power grids, financial systems, voting machines), needs to be open source. If source code is kept secret, vulnerabilities can go undetected for years. The Heartbleed vulnerability that was recently found in OpenSSL is a prime example of this – the vulnerability was present for 2 years before it was discovered, and because the source code for OpenSSL was closed (by practice), it was difficult to discover that the vulnerability existed. However, if the source code had been open, anyone with the skills and interest could have reviewed the code and found the vulnerability long before it was exploited. Therefore, security is positively correlated to the openness/transparency of the source code.

The expansion of access and availability to software is a common trend with the introduction of open-source software; despite providing an opportunity for everyone to use it, there is still a question of equity. Software is becoming the world's new economy, and access to software is dependent entirely on money when it is proprietary. Open-source software can provide the same tools to students in developing nations, researchers who don't have enough funding to be able to compute, and to allow small businesses to compete; it is through the use of Python, in particular, that has changed the way students and researchers communicate. A student will learn Python in rural India, and the student and researcher will communicate with each other in the same language, using the same libraries.

The Case Against

The counterpoint to the idea of equitable access to open-source software is that it is unrealistic in light of economic realities. The creation of high-quality software takes a significant investment of resources, time, expertise, and maintenance. Businesses have abused permissive open-source licenses for decades by taking from the community and not returning value to it. Most of the individuals creating critical open-source infrastructure are doing so as volunteers or through large companies that do not focus on the maintenance of their contributions. When a critical library is maintained by three individuals and has been downloaded millions of times, this is an indicator of a major systemic weakness.

Also, the argument of innovation incentives exists. Companies conduct research and develop new ideas to protect what they created. If software were required to be open-source, then the entire software industry would be defeated as an economic model. There are some specialized professional tools that likely would not exist if there were no longer a way to profit from the sales of the tooling. So, the question isn't whether open-source software is beneficial, because it obviously is. The question is if it works in all situations.

Is It Ethical to Build Profitable Products on Free Community Labor?

This can be a difficult question in the open-source community today. For instance, more than 90% of the world's fastest supercomputers operate using Linux along with the majority of cloud computing services and all Android devices in existence. Amazon Web Services (AWS) almost

exclusively runs on Linux and open-source applications, and generates tens of billions of dollars in revenue each year. In contrast, the budget for The Linux Foundation is significantly lower than that.

Red Hat had developed a business model (supporting its customers using enterprise variations of Linux) which IBM valued at $34 billion in 2019. Google created its online search engine along with all of its advertising infrastructure as well as its cloud computing service relying on Linux, Python, and numerous other open-source programming libraries. Meta's AI research employs Pytorch, which Meta chose to open-source, but also relies on both Python and Linux.

How ethical is it? An ethical dilemma can be assessed through various frameworks, including legally from the perspective of contractual obligations and philosophically through other more abstract frameworks. Although there are legal contracts to permit the use of the source code to create trillion-dollar businesses, there is merit to debate this situation from an ethical viewpoint. The authors and creators of the software that have bolstered the success of these trillion-dollar businesses often created those works in their spare time and were not compensated for their contributions commensurate to their value. The community of open-source developers is becoming more aware of the potential for structural inequity to exist within their community and several instances have demonstrated this inequity, whether it is around curl’s creator billing Amazon for support, maintenance issues to behalf of Heartbleed, and the chaos surrounding Log4Shell.

It is my belief that it is not unethical to use open-source software commercially, but it is unethical to not contribute if you can afford to do so. Oregon State University is a great example of an organization that is already contributing, mainly as a result of the influence exerted by the open-source community. Similarly, Google provides sponsorship for many of the open-source projects that they benefit from. However, it is an expectation for any organization that is benefiting financially from an open-source project to contribute back in the form of code, funding, documentation, and general community support.

Standing on the Shoulders of Giants

The phrase "I am standing on the shoulders of giants" applies to software more literally than anywhere else, and you can see how true this is by considering what happens every time you write your own lines of Python, which is interpreted with an interpreter that has been built over a 30-year period by thousands of different people, all of them contributing to that interpreter. That interpreter runs on a set of operating systems whose source code has been developed over the last few decades, from millions of lines of source code, and whose compiler was built with GCC (which is open source). The networking stack is also due to the use of TCP/IP, which was developed as research at an academic institution and is now available for free to anyone who wants to use it. If you are using Python to do your database queries, the database management system is probably PostgreSQL or MySQL, both of which are open source.

Open source increases innovation, rather than inhibiting it, by offering next-generation builders a higher place from which to begin building. For instance, when developing a website today, a developer does not have to create a web server from scratch. Instead, they can access Nginx, which was built using open source software just like every other project in the computer world today. Likewise, when creating machine learning libraries, developers do not have to start from scratch;

instead, they can use either TensorFlow or PyTorch, both of which use open source libraries also. Likewise, there is no need for developers to develop their own databases from scratch; they can use open source databases such as PostgreSQL. Because each of these examples is based on an established, well-documented foundation, there are fewer obstacles to the pace of innovation accomplished via each of these projects.

5. Part B — Linux Footprint

Currently, python can be easily installed on any distribution of linux due to the pre-installed copy that comes standard with most modern distributions. In those few instances where there is no default pre-installed copy of python available, you can simply install using your distribution's package manager. This section will provide an overview of the linux footprint for python — how python resides in the linux operating system and its impact upon computer security, as well as how to manage the installation of python.

Installation Method

Python's installation methods on Linux systems include:

- Debian/Ubuntu: `sudo apt install python3 python3-pip`
- RHEL/Fedora/CentOS: `sudo dnf install python3` or `sudo yum install python3`
- Arch: `sudo pacman -S python`
- Source installation can be completed by downloading Python from python.org, then running `./configure`, `make`, `make install`.

For most users, using a package manager is the preferred method since it integrates into the overall update management system, automatically receiving security patches as part of normal system updates. Source installations give you more options for versioning and build flags, but require you to manually manage updates.

Key Directories on a Linux System

/usr/bin/python3	The main Python interpreter binary. Running 'which python3' confirms this path.
/usr/lib/python3.x/	Standard library modules. This contains all the built-in modules like os, sys, json, and re.
/usr/local/lib/python3.x/	Third-party packages installed via pip go here when installed system-wide.
/etc/python3/	System-wide Python configuration (if present on the distribution).
~/.local/lib/python3.x/	Per-user package installations. Using pip install --user installs here.
/var/log/	Python itself does not write logs here by default, but applications using Python's logging module commonly write to /var/log/appname.log

User and Group — Why It Matters for Security

Anytime a Python script is executed it is done so as the user who started the process, which follows the principle of least privilege. Thus when running a Python web app as root any vulnerabilities in the app (such as code injection vulnerabilities or deserialization issues) would also have the privileges of root, and therefore an attacker could take control of your entire system.

Best practice for creating a Python app is to make an application specific service user. An instance of this practice may include running a web app as www-data (or what typically is used for default apache/nginx users on debian) or creating a custom service user with no login shell. Service users should have read access only to what code they need to execute and write access only for directories that are necessary (e.g; logs, upload).

Service Management — Running Python as a Service

Modern Linux systems use systemd to manage services. A Python application — for example, a Flask web server — can be run as a persistent service with the following unit file:

```
[Unit]Description=My Python AppAfter=network.target[Service]User=pythonappExecStart=/usr/bin/python3
/opt/myapp/app.pyRestart=on-failure[Install]WantedBy=multi-user.target
```

With this file at /etc/systemd/system/myapp.service, the commands are:

- sudo systemctl start myapp — Start the service
- sudo systemctl stop myapp — Stop the service
- sudo systemctl restart myapp — Restart after a code change
- sudo systemctl enable myapp — Enable at boot
- sudo systemctl status myapp — Check if it is running

Update Model — How Security Patches Reach Your System

The security update model in Python has two paths: the interpreter and third-party libraries. The Python Software Foundation releases regular maintenance versions (ex., 3.11.7, 3.11.8) with security fixes for the interpreter, and when installing Python via a distribution's package manager, these security updates will be included in the updates that you receive through that manager. Therefore, you can run either apt upgrade or dnf upgrade to get security updates for the Python interpreter depending on the distribution you are using.

For third-party libraries that you install with pip, security updates will occur differently than the way they occur for the interpreter. You can run pip install --upgrade package_name to upgrade a particular package, and running pip list --outdated will show you a list of packages that have an upgrade available. There are also tools (such as pip-audit) to scan installed packages to see if any have been assigned a CVE (Common Vulnerabilities and Exposures) and are, therefore, vulnerable. This screening is very important for any system used in production environments.

6. Part C — The FOSS Ecosystem

Dependencies: What Python Is Built On

The most basic level of Python (which is implemented as cpython or c python) is written in c and relies on both the c standard library and the GNU compiler collection (GCC) or the clang compiler for compilation. It uses GNU make as part of its build system. On Linux, Python uses calls to the kernel of the operating system through system calls provided by glibc (the GNU c library) which itself is an example of an important open source project.

The key dependencies visible to end users are:

- OpenSSL — provides the cryptographic primitives behind Python's ssl module and secure HTTPS connections
- zlib — compression library used by Python's gzip and zipfile modules
- libbz2, liblzma — additional compression libraries
- libffi — allows Python to call C functions dynamically (essential for ctypes)
- readline — provides command-line editing in the Python REPL
- sqlite3 — Python ships with a built-in sqlite3 module for database access

What Python Has Enabled: The Ecosystem It Created

The open-source nature of Python has allowed for the creation of an amazing set of tools. Below are just a few examples of what has been built with it:

Scientific Computing

NumPy (originally Numeric, 1995, later renamed in 2006), has introduced efficient array operations to the Python programming language, which has opened opportunities for computational science to thrive. The creation of the SciPy library built on NumPy to provide algorithms for numerical integration, optimization, and statistics. The Matplotlib package provided plotting capabilities. Collectively these three open-source tools built on Python have provided the foundation for computational science; the gravitational wave detection by the researchers at LIGO in 2015 was accomplished by analysis using all three libraries, in addition to Python.

Machine Learning and AI

To meet the growing interest in deep learning in 2015 and beyond, Google created TensorFlow and Meta created PyTorch, both of which are open-source deep learning frameworks built on top of Python. Other open-source libraries built on top of Python are Scikit-learn (which provides classical machine learning algorithms) and the Hugging Face transformers library (which powers most contemporary Natural Language Processing). The massive amount of artificial intelligence research throughout the 2020s relies on Python.

Web Development

There are many many other open-source frameworks to build web applications with Python. Django is an open-source web framework created in 2005 that supports applications for companies such as Instagram, Pinterest, and Mozilla. FastAPI is a newer Python web framework, released in 2018, that

has become the defacto standard for building high-performance APIs with Python. There is an impressive number of web frameworks built on top of Python.

Automation and DevOps

Numerous IT automation tools, including Ansible (open-source) (used by NASA, Volkswagen, and thousands of companies worldwide), Fabric, Salt, and many others are built on top of Python.

Python in the LAMP Stack / Web Context

While the original LAMP web stack (Linux/Apache/MySQL/PHP) predates the emergence of Python as the dominant web programming language, the modern version of this web stack has been referred to as LAPP (Linux/Apache (or Nginx)/PostgreSQL/Python) because Python has replaced PHP. This is a typical modern Python-related web development tech stack:

Layer	Component
Operating System	Linux (Ubuntu, Debian, RHEL — all open source)
Web Server	Nginx or Apache (both open source) reverse-proxying to Python
WSGI/ASGI Server	Gunicorn or Uvicorn — open source Python servers
Application	Django, Flask, or FastAPI — open source Python frameworks
Database	PostgreSQL or MySQL — both open source
Cache	Redis — open source in-memory data store

Community: Who Governs Python?

Python is operated by the Python Software Foundation (PSF), created in 2001 and is a nonprofit association. The PSF is the holder of the intellectual property of Python, such as the trademark and the licence. The foundation is responsible for funding the development of Python, facilitating events for the community (such as PyCon events held around the world), and managing the Python Package Index (PyPI), which is a repository that houses more than 500,000 packages from other vendors.

The technical governance of Python is handled by a five-member Steering Council that is elected by the core developers of Python. This structure replaced the earlier governing body of Python, the "Benevolent Dictator for Life" (BDFL), when Guido van Rossum, who served as the BDFL for Python since 1991, resigned from his position in 2019. This transition went smoothly due to the maturity of the Python community.

Some forms of communities associated with developing and using Python include the Python-Dev mailing list for programming language development, https://discuss.python.org, which is where most discussions take place, and the Stack Overflow forum, where Python is one of the most searched programming languages. Each year at PyCon US and at other regional PyCon events, the Python community meets in person.

7. Part D — Open Source vs Proprietary

Python's closest proprietary alternative in the scientific and engineering computing space is MATLAB, developed by MathWorks. MATLAB is a high-level language with a strong numerical computing emphasis, a well-established toolbox ecosystem, and a long history in engineering academia and industry. The comparison below is honest — there are areas where MATLAB genuinely exceeds Python, and areas where Python's open-source nature is an insurmountable advantage.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free forever — zero licensing fees	Approx. $2,000–$5,000/year per license
Security (code audit)	Full source available; community finds and patches vulnerabilities openly	Closed source; trust vendor's security claims only
Support & Reliability	PSF, Stack Overflow, forums, paid support via vendors; 30+ years of stability	MathWorks provides commercial support; SLAs available
Freedom to Modify	Full freedom — fork, patch, redistribute as you wish under PSF license	No modification; locked to vendor's roadmap
Community vs Corporate	Governed by PSF; thousands of contributors; no single entity controls it	Controlled entirely by MathWorks; decisions are proprietary
Overall Verdict	Recommended for nearly all use cases; enormous ecosystem and no lock-in	Better only for legacy engineering workflows tied to MATLAB toolboxes

Verdict — Would I Deploy Python in a Real-World Context?

MATLAB, developed by MathWorks has become the only significant proprietary competitor to Python in the scientific and engineering computing space. MATLAB is a high-level programming language that was designed for numerical computing, has a mature toolbox ecosystem, and has been around longer in engineering universities and industry, than Python has. The below comparison of MATLAB vs. Python is an honest evaluation; there are areas where MATLA does indeed have advantages over Python, and there are also instances where Python's open-source nature affords its advantages over MATLAB that cannot be overcome.

Absolutely! I would use Python for almost any software project. Today, the Python ecosystem is so comprehensive that there is very little chance you will find an area of interest where there isn't a Python library for it. Because it is open source, licensing to deploy it commercially is not a concern. Its package manager (pip) and virtual environment tools (venv, conda) make it reasonable to manage dependencies. Finally, Python has an extremely large, active, and well-documented community.

Unless you find yourself in a legacy engineering organisation that has existing MATLAB code, or need to use a MathWorks tool (like Simulink for model-based design) that doesn't have a corresponding Python implementation, then Python is a better choice in every respect than MATLAB. Will I be contributing to developing Python? Yes, I already have plans to do so. The core CPython project welcomes contributions via an established process on GitHub as well as documentation enhancements of all kinds since all open-sources comprise contributions to the community, which provides plenty of opportunity for contributions from all sources related to Python.

8. Shell Script Tasks

Five shell scripts accompany this report. Each demonstrates practical Linux skills connected to the open-source software examined above.

Script 1 — System Identity Report

This script produces a welcome screen for the Linux system, displaying kernel version, current user, home directory, system uptime, and the open-source license of the OS.

Concepts used: variables, command substitution with $(), echo with formatting, uname, whoami, uptime.

```
This message is shown once a day. To disable it please create the
/home/devesh/.hushlogin file.
devesh@LAPTOP-7T9PVSAU:~$ nano script1.sh
devesh@LAPTOP-7T9PVSAU:~$ chmod +x script1.sh
devesh@LAPTOP-7T9PVSAU:~$ ./script1.sh
====================================
 Open Source Audit - devesh
====================================
Distro    : Ubuntu 24.04.4 LTS
Kernel    : 6.6.87.2-microsoft-standard-WSL2
User      : devesh
Home Dir  : /home/devesh
Uptime    : up 3 hours, 7 minutes
Date/Time : Sun Mar 29 13:50:18 UTC 2026
OS License: GNU General Public License v2
Software  : Python - PSF License
====================================
devesh@LAPTOP-7T9PVSAU:~$
```

Script 2 — FOSS Package Inspector

Checks whether Python3 is installed on the system, displays its version and license, and uses a case statement to print a philosophy note about the chosen package.

Concepts used: if-then-else, case statement, dpkg -l or rpm -q, pipe with grep.

```
devesh@LAPTOP-7T9PVSAU:~$ nano script2.sh
devesh@LAPTOP-7T9PVSAU:~$ nano script2.sh
devesh@LAPTOP-7T9PVSAU:~$ chmod +x script2.sh
devesh@LAPTOP-7T9PVSAU:~$ ./script2.sh
python3 is installed.
Version: Python 3.12.3
Package: ii  libpython3-stdlib:amd64       3.12.3-0ubuntu2.1
hon3 version)
Python: 30 years of open collaboration that powered the AI revolution
devesh@LAPTOP-7T9PVSAU:~$
```

Script 3 — Disk and Permission Auditor

Loops through key system directories, reports permissions and disk usage for each, and checks for Python's library directory specifically.

Concepts used: for loop, arrays, du, ls -ld, awk, cut, directory existence test with -d.

```
devesh@LAPTOP-7T9PVSAU:~$ nano script3.sh
devesh@LAPTOP-7T9PVSAU:~$ chmod +x script3.sh
devesh@LAPTOP-7T9PVSAU:~$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxr-xr-x root syslog | Size: 367M
/home => Permissions: drwxr-xr-x root root | Size: 56K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 28K

--- Python Library Check ---
Found: /usr/lib/python3
Permissions: drwxr-xr-x root root
devesh@LAPTOP-7T9PVSAU:~$
```

Script 4 — Log File Analyzer

Reads a log file line by line, counts occurrences of a keyword, and prints the last five matching lines. Accepts the log path and keyword as command-line arguments.

Concepts used: while-read loop, if-then inside loop, counter variables, command-line arguments ($1, $2), file existence check, grep, tail.

```
devesh@LAPTOP-7T9PVSAU:~$ chmod +x script4.sh
devesh@LAPTOP-7T9PVSAU:~$ nano test.log
devesh@LAPTOP-7T9PVSAU:~$ ./script4.sh test.log error
Keyword 'error' found 2 times in test.log

--- Last 5 matching lines ---
ERROR: Something went wrong
ERROR: Connection failed
```

Script 5 — Open Source Manifesto Generator

Asks the user three interactive questions and generates a personalised open-source philosophy statement, saving it to a dated text file.

Concepts used: read for interactive input, string concatenation, writing to a file with >, date command, cat to display output.

```
devesh@LAPTOP-7T9PVSAU:~$ nano script5.sh
devesh@LAPTOP-7T9PVSAU:~$ chmod +x script5.sh
devesh@LAPTOP-7T9PVSAU:~$ ./script5.sh
=== Open Source Manifesto Generator ===
Enter your name:
Devesh
Why do you support open source?
learning and sharing knowledge
Your favorite open-source tool:
Linux

Manifesto saved to manifesto_2026-03-29.txt

----------------------------------------
 Open Source Manifesto
----------------------------------------
Name      : Devesh
Date      : 2026-03-29
Reason    : learning and sharing knowledge
Tool      : Linux

Statement :
I, Devesh, believe in the power of open source.
I support it because learning and sharing knowledge.
I actively use tools like Linux to learn, share, and grow.
----------------------------------------
```

9. References

On Python's History and Philosophy

- van Rossum, G. (1991). Python 0.9.0 release notes. alt.sources newsgroup.
- van Rossum, G. (2009). The History of Python blog series. python-history.blogspot.com
- Python Software Foundation. (2024). About Python. python.org/about
- Python Software Foundation. (2024). PSF License. python.org/psf/license

On Open Source Philosophy and Licensing

- Stallman, R. (2002). Free Software, Free Society. GNU Press.
- Raymond, E.S. (1999). The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Open Source Initiative. (2024). The Open Source Definition. opensource.org/osd
- SPDX License List. (2024). PSF-2.0. spdx.org/licenses/PSF-2.0.html
- choosealicense.com. (2024). MIT License. choosealicense.com/licenses/mit

On Linux and Shell Scripting

- Shotts, W. (2019). The Linux Command Line, 2nd Edition. No Starch Press. linuxcommand.org
- GNU Project. (2024). GNU Bash Manual. gnu.org/software/bash/manual
- Linux Foundation. (2024). Linux Kernel Documentation. kernel.org/doc

On the Python Ecosystem

- Python Software Foundation. (2024). Python Package Index (PyPI). pypi.org
- NumPy Developers. (2024). NumPy About. numpy.org/about
- Django Software Foundation. (2024). Django Documentation. djangoproject.com
- LIGO Scientific Collaboration. (2016). GW150914: First direct detection of gravitational waves. Physical Review Letters.

On the Ethics of Open Source

- Eghbal, N. (2016). Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure. Ford Foundation.
- Heartbleed.com. (2014). The Heartbleed Bug. heartbleed.com
- Red Hat / IBM. (2019). IBM to Acquire Red Hat. ibm.com/news